

Iterating through all masks with their submasks. $O(3^n)$

```
for (int m=0; m<(1<<n); ++m)
    for (int s=m; s; s=(s-1)&m)
        ... s and m ...
```

bridges.cpp

```
void Graph::bridgeUtil(int u, vector<bool>& visited,
vector<int>& disc, vector<int>& low, int parent) {
    static int time = 0;
    visited[u] = true;
    disc[u] = low[u] = ++time;
    list<int>::iterator i;
    for (i = adj[u].begin(); i != adj[u].end(); ++i) {
        int v = *i;
        if (v==parent)
            continue;
        if (visited[v]) {
            low[u] = min(low[u], disc[v]);
        } else {
            parent = u;
            bridgeUtil(v, visited, disc, low, parent);
            low[u] = min(low[u], low[v]);
            if (low[v] > disc[u])
                cout << u << " " << v << endl; // a bridge
        }
    }
}

void Graph::bridge()
{
    vector<bool> visited (V,false);
    vector<int> disc (V,-1);
    vector<int> low (V,-1);

    int parent = -1;

    // Call the recursive helper function to find Bridges
    // in DFS tree rooted with vertex 'i'
    for (int i = 0; i < V; i++)
        if (visited[i] == false)
            bridgeUtil(i, visited, disc, low, parent);
}
```

combinatorics.cpp

```
class CombPrime;
constexpr int MAXN = 200'001;
constexpr int MAXMOD = 1e6;
map<int, CombPrime> prime_comb_map;

long long ext_euclidean(long long a, long long b, long long& x, long long& y) {
    x = 1, y = 0;
    long long x1 = 0, y1 = 1, a1 = a, b1 = b;
    while (b1) {
        long long q = a1 / b1;
        tie(x, x1) = make_tuple(x1, x - q * x1);
        tie(y, y1) = make_tuple(y1, y - q * y1);
        tie(a1, b1) = make_tuple(b1, a1 - q * b1);
    }
    return a1;
}

long long mod_inv(long long n, long long _mod) {
    long long x, y;
    ext_euclidean(n, _mod, x, y);
    return (x % _mod + _mod) % _mod;
}

long long mod_exp(long long base, long long exp, long long _mod) {
    long long ans = 1;
    base %= _mod;
    while (exp > 0) {
        if (exp & 1) ans = (base * ans) % _mod;
        exp /= 2;
        base = (base * base) % _mod;
    }
    return ans;
}

class CombLargePrime {
public:
    const long long MOD;
    vector<long long> fact, invfact;
```

```
CombLargePrime(long long _MOD, int compute = MAXN) :
    MOD(_MOD),
    fact(vector<long long>(compute + 1, 1)),
    invfact(vector<long long>(compute + 1, 1)) {

    for (int i = 1; i <= compute; i++) {
        fact[i] = (i * fact[i - 1]) % MOD;
        invfact[i] = (mod_exp(i, MOD - 2, MOD) * invfact[i - 1]) % MOD;
    }

    long long comb(int N, int K) const {
        return fact[N] * invfact[K] % MOD * invfact[N - K] % MOD;
    }
};

class CombPrime {
public:
    const long long p, b, MOD;
    vector<long long> p_cnt, powers, fact, invfact;
    CombPrime() : p(-1), b(-1), MOD(-1) {}
    CombPrime(long long _p, int compute = MAXN) :
        p(_p),
        b(floor(log2(MAXMOD + 1) / log2(_p))),
        MOD(powl(_p, b)),
        p_cnt(vector<long long>(compute + 1, 0)),
        powers(vector<long long>(b + 1, 1)),
        fact(vector<long long>(compute + 1, 1)),
        invfact(vector<long long>(compute + 1, 1)) {

        for (int i = 1; i <= b; i++) {
            powers[i] = p * powers[i - 1];
        }

        for (int i = 1; i <= compute; i++) {
            p_cnt[i] = p_cnt[i - 1];

            int x = i;
            while (x % p == 0) p_cnt[i]++, x /= p;
        }
    }
};
```

```
fact[i] = (x * fact[i - 1]) % MOD;
invfact[i] = (mod_inv(x, MOD) * invfact[i - 1]) % MOD;
}

long long comb(int N, int K, int exp = 1) const {
    if (p_cnt[N] - p_cnt[K] - p_cnt[N - K] >= exp) return 0LL;
    return fact[N] * invfact[K] % powers[exp] * invfact[N - K] % powers[exp];
}

class CombCompound {
public:
    vector<pair<int, int>> factors;
    int f_sz;
    CombCompound(int N, int compute = MAXN) {
        for (long long i = 2; i * i <= N; i++) {
            if (N % i != 0) continue;
            int exp = 0;
            while (N % i == 0) {
                exp++;
                N /= i;
            }
            factors.push_back({(int) i, exp});
        }
        if (N > 1) factors.push_back({N, 1});
        for (auto& factor : factors) {
            if (!prime_comb_map.count(factor.first)) {
                prime_comb_map.emplace(factor.first, CombPrime(factor.first));
            }
        }
        f_sz = factors.size();
    }

    long long comb(int N, int K) const {
        if (f_sz == 0) return 0;
        long long a1, m1;
        for (int i = 0; i < f_sz; i++) {
            long long a2 = prime_comb_map[factors[i].first].comb(N, K, factors[i].second);
            long long m2 = powl(factors[i].first, factors[i].second);
        }
    }
};
```

```
if (i == 0) {
    a1 = a2;
    m1 = m2;
} else {
    a1 = a1 * mod_inv(m2, m1) * m2 + a2 * mod_inv(m1, m2) * m1;
    m1 = m1 * m2;
    a1 %= m1;
}

return a1;
}

};
const int MOD = 1e9 + 7;
const CombLargePrime comb(MOD);
```

Catalan number

C_n is the solution for

- Number of correct bracket sequence consisting of n opening and n closing brackets.

$$C_n = \frac{1}{(n+1)} * (2n \text{ choose } n)$$

```
const int MOD = 1e9+7;
const int MAX = 1e5;
int catalan[MAX];
void init() {
    catalan[0] = catalan[1] = 1;
    for (int i=2; i<=MAX; i++) {
        catalan[i] = 0;
        for (int j=0; j < i; j++) {
            catalan[i] += (catalan[j] * catalan[i-j-1]) % MOD;
            if (catalan[i] >= MOD) {
                catalan[i] -= MOD;
            }
        }
    }
}
```

custom hash.cpp

```
struct custom_hash {
    static size_t splitmix64(long long x) {
        // http://arbitraryf.dl.unimil.it/splitmix64.c
        x = 0x9e3779b97f4a7c15;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
        x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
        return x ^ (x >> 31);
    }

    size_t operator()(long long x) const {
        static const long long FIXED_RANDOM = chrono::steady_clock::now().time_since_epoch().count();
        return splitmix64(x + FIXED_RANDOM);
    }
};
```

dsu.cpp

```
struct UnionFind {
    vector<int> parent, size;
    explicit UnionFind(int n) : parent(n), size(n, 1) {
        iota(parent.begin(), parent.end(), 0);
    }
    explicit UnionFind(const UnionFind& other) : parent(other.parent), size(other.size) {}

    int find(int node) {
        if (node == parent[node]) return node;
        return parent[node] = find(parent[node]);
    }

    bool unite(int node1, int node2) {
        int p1 = find(node1);
        int p2 = find(node2);
        if (p1 == p2) return false;
        if (size[p1] < size[p2]) swap(p1, p2);
        parent[p2] = p1;
        size[p1] += size[p2];
        return true;
    }
};
```

fenwick.cpp

```
struct FenwickTree {
    int sz;
    vector<int> bit;

    explicit FenwickTree(int c_sz) : sz(c_sz + 1), bit(c_sz + 1, 0) {}

    int query(int r) {
        int ans = 0;
        for (r++; r > 0; r -= r & -r)
            ans += bit[r];
        return ans;
    }

    int query(int l, int r) {
        return query(r) - query(l - 1);
    }

    void update(int i, int delta) {
        for (i++; i < sz; i += i & -i)
            bit[i] += delta;
    }

    void clear() {
        fill(bit.begin(), bit.end(), 0);
    }
};
```

fft.cpp

```
vector<int> multiply_ntt(vector<int> const& a, vector<int> const& b) {
    vector<int> fa(a.begin(), a.end()), fb(b.begin(), b.end());
    int n = 1;
    while (n < a.size() + b.size())
        n <<= 1;
    fa.resize(n);
    fb.resize(n);

    ntt(fa, false);
    ntt(fb, false);
    for (int i = 0; i < n; i++)
        fa[i] = 1LL * fa[i] * fb[i] % mod;
    ntt(fa, true);

    return fa;
}
```

```
// multiply two polynomials
vector<int> multiply(vector<int> const& a, vector<int> const& b) {
    vector<cd> fa(a.begin(), a.end()), fb(b.begin(), b.end());
    int n = 1;
    while (n < a.size() + b.size())
        n <<= 1;
    fa.resize(n);
    fb.resize(n);

    fft(fa, false);
    fft(fb, false);
    for (int i = 0; i < n; i++)
        fa[i] *= fb[i];
    fft(fa, true);

    vector<int> result(n);
    for (int i = 0; i < n; i++)
        result[i] = round(fa[i].real());
    return result;
}
```

```
using cd = complex<double>;
const double PI = acos(-1);
```

```
void fft(vector<cd> & a, bool invert) {
    int n = a.size();

    for (int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1)
            j ^= bit;
        j ^= bit;

        if (i < j)
            swap(a[i], a[j]);
    }

    for (int len = 2; len <= n; len <<= 1) {
        double ang = 2 * PI / len * (invert ? -1 : 1);
        cd wlen(cos(ang), sin(ang));
        for (int i = 0; i < n; i += len) {
            cd w(1);
            for (int j = 0; j < len / 2; j++) {
                cd u = a[i+j], v = a[i+j+len/2] * w;
                a[i+j] = u + v;
                a[i+j+len/2] = u - v;
                w *= wlen;
            }
        }
    }

    if (invert) {
        for (cd & x : a)
            x /= n;
    }
}

const int mod = 998244353;
const int root = 15311432;
const int root_1 = 469870224;
const int root_pw = 1 << 23;

// const int root = mod_pow(G, (MOD - 1) / ROOT_PW, MOD);
// const int root_1 = mod_inv(root, MOD);

// number theoretic theorem: under modulo 998244353
void ntt(vector<int> & a, bool invert) {
    int n = a.size();

    for (int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1)
            j ^= bit;
        j ^= bit;

        if (i < j)
            swap(a[i], a[j]);
    }

    for (int len = 2; len <= n; len <<= 1) {
        int wlen = invert ? root_1 : root;
        for (int i = len; i < root_pw; i <<= 1)
            wlen = (int)(1LL * wlen * wlen % mod);

        for (int i = 0; i < n; i += len) {
            int w = 1;
            for (int j = 0; j < len / 2; j++) {
                int u = a[i+j], v = (int)(1LL * a[i+j+len/2] * w % mod);
                a[i+j] = u + v < mod ? u + v : u + v - mod;
                a[i+j+len/2] = u - v >= 0 ? u - v : u - v + mod;
                w = (int)(1LL * w * wlen % mod);
            }
        }
    }

    if (invert) {
        int n_1 = mod_inv(n, mod);
        for (int & x : a)
            x = (int)(1LL * x * n_1 % mod);
    }
}
```

largest-zero-submatrix.cpp

```
int zero_matrix(vector<vector<int>> a) {
    int n = a.size();
    int m = a[0].size();

    int ans = 0;
    vector<int> d(m, -1), d1(m), d2(m);
    stack<int> st;
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < m; ++j) {
            if (a[i][j] == 1)
                d[j] = i;
        }

        for (int j = 0; j < m; ++j) {
            while (!st.empty() && d[st.top()] <= d[j])
                st.pop();
            d1[j] = st.empty() ? -1 : st.top();
            st.push(j);
        }

        while (!st.empty())
            st.pop();

        for (int j = m - 1; j >= 0; --j) {
            while (!st.empty() && d[st.top()] <= d[j])
                st.pop();
            d2[j] = st.empty() ? m : st.top();
            st.push(j);
        }

        while (!st.empty())
            st.pop();

        for (int j = 0; j < m; ++j)
            ans = max(ans, (i - d[j]) * (d2[j] - d1[j] - 1));
    }
    return ans;
}
```

lazysegtree-c17.cpp

```
template <class T, class Z>
struct default_apply_lazy {
    T operator()(T node, Z lazy, int len) const {
        return node + len * lazy;
    }
};

template <class T>
struct cmerge_node {
    T operator()(T a, T b) const {
        return a + b;
    }
};

template <class Z>
struct cmerge_lazy {
    Z operator()(Z a, Z b) const {
        return a + b;
    }
};

template <class T, class Z>
struct capply_lazy {
    T operator()(T node, Z lazy, int len) const {
        return node + len * lazy;
    }
};
```

```
// https://codeforces.com/blog/entry/18051
template <class T, class Z = T,
class merge_node = plus<T>,
class combine_lazy = plus<Z>,
class apply_lazy = default_apply_lazy<T, Z>>
struct LazySegmentTree {
    int n, h;
    vector<T> tree;
    vector<Z> lazy;

    T default_value;
    Z default_lazy_value;

    LazySegmentTree(int size, T def = T(), Z def_lazy = Z()) :
        default_value(def), default_lazy_value(def_lazy) {
        n = (size == 1) ? 1 : (1 << (8 * sizeof(int) - __builtin_clz(size - 1)));
        h = sizeof(int) * 8 - 1 - __builtin_clz(n);
        tree.resize(2 * n, default_value);
        lazy.resize(n, default_lazy_value);
    }

    template <typename Container>
    void build_from_container(const Container& arr) {
        copy(begin(arr), end(arr), begin(tree) + n);
        build_from_leaves();
    }

    void build_from_leaves() {
        for (int i = n - 1; i > 0; i--) tree[i] = merge_node()(tree[i < 1], tree[i < 1 | 1]);
    }

    int segment(int p) {
        return n >> (sizeof(int) * 8 - 1 - __builtin_clz(p));
    }

    void apply(int p, Z value) {
        tree[p] = apply_lazy()(tree[p], value, segment(p));
        if (p < n) lazy[p] = combine_lazy()(lazy[p], value);
    }

    void build(int p) {
        while (p > 1) {
            p >>= 1;
            tree[p] = apply_lazy()(merge_node()(tree[p < 1], tree[p < 1 | 1]), lazy[p], segment(p));
        }
    }

    void push(int p) {
        for (int s = h; s > 0; s--) {
            int i = p >> s;
            if (lazy[i] != default_lazy_value) {
                apply(i < 1, lazy[i]);
                apply(i < 1 | 1, lazy[i]);
                lazy[i] = default_lazy_value;
            }
        }
    }

    void update(int l, int r, Z value) {
        l += n, r += n + 1;
        int l0 = l, r0 = r;
        for (; l < r; l >>= 1, r >>= 1) {
            if (l & 1) apply(l++, value);
            if (r & 1) apply(--r, value);
        }
        build(l0);
        build(r0 - 1);
    }

    T query(int l, int r) {
        l += n, r += n + 1;
        push(l);
        push(r - 1);
        T resLeft = default_value, resRight = default_value;
        for (; l < r; l >>= 1, r >>= 1) {
            if (l & 1) resLeft = merge_node()(resLeft, tree[l++]);
            if (r & 1) resRight = merge_node()(tree[--r], resRight);
        }
        return merge_node()(resLeft, resRight);
    }
};
```

lca.cpp

```
void lca() { // provide n, adj
    int timer;
    vector<int> tin, tout;
    vector<vector<int>> up;
    int l = ceil(log2(n));
    {
        tin.resize(n);
        tout.resize(n);
        timer = 0;
        up.assign(n, vector<int>(l + 1));
        auto dfs = [&](auto&& dfs, int v, int p) -> void {
            tin[v] = ++timer;
            up[v][0] = p;
            for (int i = 1; i <= l; ++i)
                up[v][i] = up[up[v][i-1]][i-1];

            for (int u : adj[v]) {
                if (u != p)
                    dfs(dfs, u, v);
            }

            tout[v] = ++timer;
        };
        dfs(dfs, 0, 0);
    }
}
```

```

auto is_ancestor = [&](int u, int v) -> bool {
    return tin[u] <= tin[v] && tout[u] >= tout[v];
};

auto lca = [&](int u, int v) -> int {
    if (is_ancestor(u, v))
        return u;
    if (is_ancestor(v, u))
        return v;
    for (int i = 1; i >= 0; --i) {
        if (!is_ancestor(up[u][i], v))
            u = up[u][i];
    }
    return up[u][0];
};

vector<int> depth(n);

auto dfs_depth = [&](auto&& self, int u, int parent, int d) -> void {
    depth[u] = d;
    for(int v : adj[u]) {
        if(v != parent) {
            self(self, v, u, d + 1);
        }
    }
};

dfs_depth(dfs_depth, 0, -1, 0);

auto shortest = [&](int u, int v) -> int {
    if(u == v) return 0;
    int ancestor = lca(u, v);
    if(ancestor == u) return depth[v] - depth[u];
    if(ancestor == v) return depth[u] - depth[v];

    return depth[u] + depth[v] - 2 * depth[ancestor];
};

```

matmult-slow.cpp

```

void matmult(const vector<vector<long long>>& A, const vector<vector<long long>>& B, vector<vector<long long>>& res) {
    assert(A[0].size() == B.size());

    int n = B.size(), rows = A.size(), cols = B[0].size();
    for(int i = 0; i < rows; i++) {
        for(int j = 0; j < cols; j++) {
            for(int k = res[i][j] - 0; k < n; k++) {
                // res[i][j] += A[i][k] * B[k][j];
                res[i][j] = (res[i][j] + A[i][k] * B[k][j]) % MOD;
            }
        }
    }
}

// O(N^3 * log n)
vector<vector<long long>> matpow(vector<vector<long long>> base, vector<vector<long long>> transformation, long long n) {
    vector<vector<long long>> tmp_base = base;
    vector<vector<long long>> tmp_transformation = transformation;
    while(n > 0) {
        if(n & 1) {
            matmult(base, transformation, tmp_base);
            swap(base, tmp_base);
        }
        n >> 1;
        matmult(transformation, transformation, tmp_transformation);
        swap(transformation, tmp_transformation);
    }
    return base;
}

```

merge-intervals.cpp

```

vector<vector<int>>& mergeIntervals(vector<vector<int>>& intervals) {
    sort(intervals.begin(), intervals.end(), [](const vector<int>& a, const vector<int>& b) {
        return a[0] < b[0];
    });
    int ressz = 1;
    for(int i = 1, intervalsSz = intervals.size(); i < intervalsSz; i++) {
        if(intervals[i][0] <= intervals[ressz-1][1]) {
            intervals[ressz-1][1] = max(intervals[ressz-1][1], intervals[i][1]);
        } else {
            intervals[ressz++] = (intervals[i]);
        }
    }
    intervals.resize(ressz);
    return intervals;
}

```

mo-algorithm.cpp

```

struct Query {
    int l, r, idx;
    int64_t key;
};

inline int64_t hilbertOrder(int x, int y) {
    constexpr uint64_t logn = 18;
    constexpr uint64_t maxn = (1ull << logn) - 1;
    uint64_t res = 0;
    for (uint64_t s = 1ull << (logn - 1); s; s >>= 1) {
        bool rx = x & s, ry = y & s;
        res = (res << 2) | (rx ? ry ? 2 : 1 : ry ? 3 : 0);
        if (lrx) {
            if (ry) x ^= maxn, y ^= maxn;
            swap(x, y);
        }
    }
    return res;
}

template <typename R>
vector<R> mo_algorithm(int n, vector<Query>& queries, auto& add_idx,
    auto& remove_idx, auto& get_answer) {
    for(Query& query : queries) {
        query.key = hilbertOrder(query.l, query.r);
    }

    sort(queries.begin(), queries.end(), [](const Query& a, const Query& b) -> bool {
        return a.key < b.key;
    });

    vector<R> answers(q);
    int l = 0, r = -1;
    for(Query& query : queries) {
        while(l > query.l) {
            l--;
            add_idx(l);
        }
        while(r < query.r) {
            r++;
            add_idx(r);
        }
        while(l < query.l) {
            remove_idx(l);
            l++;
        }
        while(r > query.r) {
            remove_idx(r);
            r--;
        }
        answers[query.idx] = get_answer(query);
    }
    return answers;
}

```

stack-utils.cpp

```

void compute(auto& nums, auto& res, const auto& cmp, bool reverse) {
    int n = nums.size();
    vector<int> stk;
    for(int i = reverse ? n-1 : 0, delta = reverse ? -1 : +1; 0 <= i && i < n; i += delta) {
        while(stk.empty() && cmp(nums[i], nums[stk.back()])) {
            res[stk.back()] = i;
            stk.pop_back();
        }
        stk.push_back(i);
    }

    void usage(auto& nums, auto& nextGreater, auto& nextSmaller, auto& prevGreater, auto& prevSmaller) {
        compute(nums, nextGreater, [](int a, int b) { return a > b; }, false);
        compute(nums, prevGreater, [](int a, int b) { return a >= b; }, true);
        compute(nums, nextSmaller, [](int a, int b) { return a < b; }, false);
        compute(nums, prevSmaller, [](int a, int b) { return a <= b; }, true);

        compute(nums, nextGreater, greater<>(), false);
        compute(nums, prevGreater, greater_equal<>(), true);
        compute(nums, nextSmaller, less<>(), false);
        compute(nums, prevSmaller, less_equal<>(), true);
    }
}

```

segtree-c17.cpp

```

template <class T>
struct cmerge_node {
    T operator() (T a, T b) const {
        return a + b;
    }
};

template <class T, class merge_node = plusT>
struct SegmentTree {
    int n;
    vector<T> tree;

    T default_value;

    SegmentTree(int size, T def = T()) : default_value(def) {
        n = size;
        while(n < size) n *= 2;
        tree.resize(2*n, default_value);
    }

    template <typename Container>
    void build(const Container& arr) {
        assert((int) arr.size() <= n);
        copy(begin(arr), end(arr), begin(tree) + n);
        build_from_leaves();
    }

    void build_from_leaves() {
        for(int i = n - 1; i > 0; i--) tree[i] = merge_node()(tree[i<1], tree[i<1|1]);
    }

    void update(int pos, T val) {
        tree[n + pos] = val;
        for(int idx = (n + pos) >> 1; idx > 0; idx >>= 1) {
            tree[idx] = merge_node()(tree[idx<1], tree[idx<1|1]);
        }
    }

    T query(int x) {
        return tree[n + x];
    }

    T query(int l, int r) {
        T resLeft = default_value, resRight = default_value;
        for(l += n, r += n + 1; l < r; l >>= 1, r >>= 1) {
            if(l & 1) resLeft = merge_node()(resLeft, tree[l+1]);
            if(r & 1) resRight = merge_node()(tree[r-1], resRight);
        }
        return merge_node()(resLeft, resRight);
    }
};

```

sparse-table-c17.cpp

```
struct SparseTable {
    int n, k;
    vector<vector<int>>> st;

    template <typename Container>
    SparseTable(const Container& array) {
        n = size(array);
        k = 31 - __builtin_clz(n);
        st = vector(k + 1, vector<int>(n));
        copy(begin(array), end(array), begin(st[0]));
        for(int i = 1; i <= k; i++)
            for(int j = 0; j + (1 << i) <= n; j++)
                st[i][j] = merge(st[i - 1][j], st[i - 1][j + (1 << (i - 1))]);
    }

    int query(int l, int r) {
        int i = 31 - __builtin_clz(r - l + 1);
        return merge(st[i][l], st[i][r - (1 << i) + 1]);
    }

    int merge(int a, int b) {
        return min(a, b);
    }
};
```

spfa.cpp

```
const int INF = 1000000000;
vector<vector<pair<int, int>>> adj;

bool spfa(int s, vector<int>& d) {
    int n = adj.size();
    d.assign(n, INF);
    vector<int> cnt(n, 0);
    vector<bool> inqueue(n, false);
    queue<int> q;

    d[s] = 0;
    q.push(s);
    inqueue[s] = true;
    while (!q.empty()) {
        int v = q.front();
        q.pop();
        inqueue[v] = false;

        for (auto edge : adj[v]) {
            int to = edge.first;
            int len = edge.second;

            if (d[v] + len < d[to]) {
                d[to] = d[v] + len;
                if (!inqueue[to]) {
                    q.push(to);
                    inqueue[to] = true;
                    cnt[to]++;
                    if (cnt[to] > n)
                        return false; // negative cycle
                }
            }
        }
    }
    return true;
}
```

```
struct Edge {
    int a, b, cost;
};
```

```
int n;
vector<Edge> edges;
const int INF = 1000000000;
```

```
void solve() {
    vector<int> d(n, 0);
    vector<int> p(n, -1);
    int x;

    for (int i = 0; i < n; ++i) {
        x = -1;
        for (Edge e : edges) {
            if (d[e.a] + e.cost < d[e.b]) {
                d[e.b] = max(-INF, d[e.a] + e.cost);
                p[e.b] = e.a;
                x = e.b;
            }
        }
    }
}
```

```
if (x == -1) {
    cout << "No negative cycle found.";
} else {
    for (int i = 0; i < n; ++i)
        x = p[x];

    vector<int> cycle;
    for (int v = x;; v = p[v]) {
        cycle.push_back(v);
        if (v == x && cycle.size() > 1)
            break;
    }
    reverse(cycle.begin(), cycle.end());

    cout << "Negative cycle: ";
    for (int v : cycle)
        cout << v << ' ';
    cout << endl;
}
}
```

tarjan.cpp

```
// get all Strongly Connected Components (SCC) where
// you can reach every node from every other node in a Directed Graph
vector<vector<int>> tarjan(vector<vector<int>>& adj) {
    int n = adj.size();
    vector<vector<int>>> res;

    int index = 0;
    vector<int> S;

    vector<int> v_index(n, -1), v_lowlink(n, -1);
    vector<bool> v_onStack(n);

    auto strongconnect = [&](auto&& self, int v) -> void {
        v_index[v] = v_lowlink[v] = index;
        index++;
        S.push_back(v);
        v_onStack[v] = true;

        for (int w : adj[v]) {
            if (v_index[w] == -1) {
                self(self, w);
                v_lowlink[v] = min(v_lowlink[v], v_lowlink[w]);
            } else if (v_onStack[w]) {
                v_lowlink[v] = min(v_lowlink[v], v_index[w]);
            }
        }

        if (v_lowlink[v] == v_index[v]) {
            res.emplace_back();
            int w;
            do {
                w = S.back();
                S.pop_back();
                v_onStack[w] = false;
                res.back().push_back(w);
            } while (w != v);
        }
    };

    for (int v = 0; v < n; v++) {
        if (v_index[v] == -1) {
            strongconnect(strongconnect, v);
        }
    }
    return res;
}
```

reroot.cpp

```
namespace reroot {
    const auto exclusive = [](const auto& a, const auto& base, const auto& merge_into, int vertex) {
        int n = (int)a.size();
        using Aggregate = std::decay_t

```

```
int n, m;
vector<vector<pair<int, int>>> g;
vector<int> path;
vector<bool> seen;

void dfs(int node) {
    while (!g[node].empty()) {
        auto [son, idx] = g[node].back();
        g[node].pop_back();
        if (seen[idx]) { continue; }
        seen[idx] = true;
        dfs(son);
    }
    path.push_back(node);
}

// EXAMPLE
dfs(0);

if (path.size() != m + 1) {
    cout << "IMPOSSIBLE";
} else {
    for (int node : path) { cout << node + 1 << ' '; }
}
```

acc-1117 : pantomime-staple-chimp

Josephus Problem

$$J_{n,k} = ((J_{n-1,k} + k - 1) \bmod n) + 1$$

$$J_{1,k} = 1$$

Eulerian path/trail